

React

```
import { useState } from "react";
import "./App.css";
import Component from "./Component";

function App() {
  const [count, setCount] = useState(10);

  const handleClick = () => {
    setCount(count + 1);
    // count+1;
  };

  return (
    <>
      { /* jsx */}
      <h1> {count} </h1>
      <button onClick={handleClick}>Count++</button>
      <h2>Hello there</h2>

      <Component name="FIIT" age = "20" />

      { /* jsx */}
    </>
  );
}

export default App;
```

```
import React, { useEffect, useState } from "react";
import axios from "axios";

const Component = ({ name, age }) => {
  // console.log(props);

  const [data, setData] = useState([]);

  const fetchData = async () => {
    const result = await axios.get(
      "https://jsonplaceholder.typicode.com/todos",
    );
    // console.log(result.data);
    setData(result.data);
  };
};
```

```

// side effect
useEffect(() => {
  fetchData();
  // console.log("hello there");
}, []);
// no depdency => runs on every render
// [] => runs only on mount
// [count] => runs when that state change

return (
  <div>
    <h1>dfsadfadsfsdfad</h1>
    <h1>{name}</h1>
    <h2>{age}</h2>

    { /* list rendering */ }

    <div
      style={{
        display: "flex",
        justifyContent: "center",
        alignItems: "center",
        flexWrap: "wrap",
      }}
    >
      {data.map((ele) => {
        return (
          <div
            key={ele.id}
            style={{
              width: "200px",
              height: "200px",
              border: "2px solid black",
              color: "white",
              margin: "20px",
              padding: "20px",
              backgroundColor: "#333",
            }}
          >
            <h1>{ele.title}</h1>
          </div>
        );
      })}
    </div>
  </div>
);
};

```

```
export default Component;
```

Theory

1. State Management

- **useState** → manages component-level state.
- Example:

```
const [count, setCount] = useState(10);
```

- Updating state triggers re-render.

2. Props

- Props allow data to be passed from parent to child.
- Example:

```
<Component name="FIIT" age="20" />
```

- In child:

```
const Component = ({ name, age }) => { ... }
```

3. useEffect (Side Effects)

- **Definition:** Runs code after render.
- **Dependency array:**
 - [] → runs once on mount.
 - [state] → runs when state changes.
 - No array → runs on every render.
- Common use: API calls, subscriptions, DOM updates.

4. Axios Integration

- **axios.get(url)** → fetch data.
- **axios.post(url, body)** → create data.
- **axios.put(url, body)** → update data.

- **axios.delete(url)** → delete data.



Coding Round Questions

- **useState** → How does useState work?
- **Props** → What are props and how are they different from state?
- **useEffect** → How does the dependency array affect execution?
- **Axios** → How do you perform CRUD operations with Axios?
- **List rendering** → How do you render lists in React?
- **Error handling** → How do you handle API errors with Axios?



Machine Coding Round Challenge

Problem: Todo App with API

Requirements:

1. Create a parent App with a counter and a child TodoComponent.
2. Fetch todos from API (jsonplaceholder.typicode.com/todos).
3. Display todos in styled cards.
4. Add new todo via POST.
5. Update todo via PUT.
6. Delete todo via DELETE.